

PETSc Rewrite of Slope Stability

From the legacy MATLAB workflow to the current PETSc runtime

- Translate the original MATLAB mental model into the current executable PETSc/Python path.
- Show where parallel ownership, PMG, unified runners, unified meshes, and unified visualisation changed the design.

Repository-local material only. Snapshot date: April 9, 2026.

- I start from the legacy MATLAB scripts, because that is still the fastest shared mental model.
- I keep one concrete architecture anchor throughout: the checked-in 3D heterogeneous SSR benchmark that exercises the current P4/PMG path.
- Later I switch deliberately to the committed P2 performance study, so architecture and runtime evidence never get mixed.

1. MATLAB baseline
2. Prerequisites and getting running
3. Structural redesign for parallelism
4. Main new functionality
5. Unified runners
6. Unified visualisation
7. Unified meshes
8. Speed comparison
9. Close and appendix

What I Use As Evidence

Legacy MATLAB

3D_hetero_SSR.m
+CONTINUATION/*
+NEWTON/*
+CONSTITUTIVE_PROBLEM/*
+MESH/*
+VIZ/*

PETSc Rewrite

README.md
run_config.py
run_case_from_config.py
run_3D_hetero_SSR_capture.py

Docs And Studies

petsc_matlab_map
config_scheme_3d.md
config_case_matrix.md
performance_comparison

How I use them together

The MATLAB tree gives the original mental model, the runtime code gives the current executable truth, and the docs package gives the shortest path through the mapping and the benchmark protocol. I use all three, but I do not substitute one for another.

MATLAB Baseline

- One benchmark is usually one top-level script.
- `slope_stability_3D_hetero_SSR.m` declares the case, chooses the solver, launches continuation, and plots the results.
- The numerical kernels are modular, but orchestration remains case-local rather than registry-driven.
- The reader learns the workflow by reading the script from top to bottom.

quadrature
→ local basis
→ mesh load
→ material expansion
→ K_{elast} , B , f_V
→ constitutive object
→ SSR continuation
→ MATLAB figures

Package	Role	Typical content
+ASSEMBLY	FE preprocessing	Quadrature, local basis, elastic stiffness, volume forces, material expansion
+CONTINUATION	Outer branch tracking	Direct SSR, indirect SSR, LL, init phase, omega stepping
+NEWTON	Nonlinear solve core	Damped Newton, indirect SSR Newton, LL Newton
+CONSTITUTIVE_PROBLEM	Stress and tangent model	Davis reduction, stress evaluation, tangent assembly, potential
+LINEAR_SOLVERS	Linear solver factory	Direct, AGMG, ICHOL, HYPRE BoomerAMG, DFGMRES, collectors
+MESH, +VIZ, +SEEPAGE	Domain-specific helpers	Case loaders, plotting, seepage preprocessing and flow solve

Accepted-step lifecycle

- Continuation chooses the next ω target.
- A secant-style predictor proposes (U, λ) .
- `NEWTON.newton_ind_SSR` solves the coupled equilibrium system.
- On success, history arrays grow and the deflation basis is updated.
- On failure, the step is shrunk and retried.

Main data objects

- `coord`, `elem`, `surf`, `Q`
- K_{elast}, B, f_V
- Integration-point material fields: c_0, ϕ, ψ, γ
- History outputs: $\lambda, \omega, u_{\text{max}}$, work counters
- Linear solver object copied into continuation calls

The MATLAB Constitutive Object Is The Main Stateful Kernel Wrapper

- `CONSTITUTIVE_PROBLEM.CONSTITUTIVE` stores the global strain-displacement matrix B , quadrature weights, material fields, and runtime measurements.
- It performs Davis reduction for a given λ , evaluates stress, evaluates the consistent tangent, and assembles residual and tangent objects.
- In other words, MATLAB centralises constitutive state in one handle object but feeds it with globally assembled FE operators.

```
reduction( $\lambda$ )  
→ stress( $U$ )  
→ stress+tangent( $U$ )  
→ build  $F$   
→ build  $F, K_{\text{tangent}}$   
→ return timings
```

Mesh Handling In MATLAB Is Closely Coupled To Element Degree

MATLAB path	What it assumes	Consequence
load_mesh_P2.m	Quadratic tetrahedral HDF5 input	Benchmark meshes and FE degree are tied together
load_mesh_gmsh_waterlevels.m	Case-specific Gmsh seepage conventions	Loader logic lives with the case family
midpoints_P2.m, midpoints_P4.m	Degree-specific node generation	Higher order support is added through dedicated utilities
Q mask from face labels	Boundary conditions embedded in the loader path	Free-DOF extraction depends on the loaded mesh convention

This is one of the biggest conceptual changes in the rewrite: the PETSc side moves toward **canonical mesh family storage** plus **solver-side degree selection**, instead of one file format per numerical path.

What the MATLAB flow still does well

- The benchmark script can call `VIZ.*` immediately after the solve and inspect the same assembled state.
- That keeps the familiar 3D and slice figures close to the benchmark logic the original authors already know.

What stays coupled

state	live MATLAB workspace after the solve
plots	benchmark-local <code>VIZ.*</code> calls and figure setup
reuse	copy-and-adapt workflow between benchmark families

solve benchmark

- call `VIZ.*`
- inspect figure state
- export figure manually
- repeat per case family

In the rewrite I keep the figure family as an outcome, but I stop depending on case-local plotting code and live interpreter state to rebuild it.

Prerequisites And Getting Running

```
$ ./bootstrap.sh  
$ BOOTSTRAP_MODE=wheel ./bootstrap.sh
```

- Default bootstrap creates `.venv`, builds PETSc with HYPRE under `.build`, installs `petsc4py`, and installs the project in editable mode.
- The wheel mode is lighter, but it may omit the full benchmark-capable PETSc/HYPRE stack.
- The repository is designed around a real local PETSc build, not just pure-Python smoke execution.

What the heavy first run buys you

`petsc4py`
HYPRE-enabled PETSc
editable install of `slope_stability`
benchmark partitioning extras
notebook-ready environment

Single case from a config

```
$ ./venv/bin/python -m  
slope_stability.cli.run_case_from_config  
benchmarks/run_3D_hetero_SSR_capture/case.toml  
-out_dir /tmp/ssr_run
```

Canonical suite

```
$ ./venv/bin/python -m  
slope_stability.cli.run_benchmark_suite
```

- A benchmark folder can also be run locally with its `run.sh`.
- The canonical MATLAB-parity subset is the set of folders whose `[benchmark].suite = true`.
- This unifies direct execution, suite execution, and notebook reuse around the same `case.toml`.

Devcontainer

- The repository ships a prebuild-friendly `.devcontainer`.
- Slow setup can be absorbed into Codespaces prebuilds.
- Validation entry point: `bash .devcontainer/validate.sh -imports-only`

Outputs every config-driven run can write

- `exports/run_debug.h5`
- `exports/continuation_history.json`
- `exports/final_solution.vtu`
- `exports/resolved_config.toml`

The rewrite standardises what MATLAB often kept as script-local variables or ad hoc figure state: the final mesh, the final fields, the continuation history, and the resolved runtime configuration.

Benchmark Folders Are The User-Facing Contract

Item	Required?	Purpose
case.toml	yes	Runtime configuration and benchmark metadata
run.sh	yes	Local convenience entry point
README.md	yes	Human-facing explanation of the case
[benchmark] section	yes	Title, MATLAB script link, comparison kind, suite inclusion, MPI ranks
[notebook] section	optional but important	Controls the generated simulation and visualisation notebooks
artifacts/simulation/*	generated	Reusable outputs for notebooks and postprocessing

Structural Redesign For Parallelism

Responsibility Map: MATLAB Driver To PETSc Modules

Role	MATLAB location	PETSc location
Case declaration	Top of the benchmark script	case.toml + run_config.py
Runner dispatch	Implicit in the script name	run_case_from_config.py
Mesh and material preprocessing	Script body + +MESH/+ASSEMBLY	mesh/* + fem/* + runner setup
Constitutive handle construction	CONSTITUTIVE.m	problem.py:ConstitutiveOperator
Continuation launch	SSR_indirect_continuation.m	continuation/indirect.py
Linear solver factory	set_linear_solver.m	solver.py:SolverFactory

This Is The Architecture Anchor I Keep Coming Back To

The benchmark I use as the walk-through case

I use the checked-in 3D heterogeneous SSR default benchmark as the architecture anchor because it keeps the owned-row, overlap, pmg_shell, and P4 path in one place:

```
analysis = ssr                                elem_type = P4
node_ordering = block_metis                    mpi_distribute_by_nodes = true
constitutive_mode = overlap                    tangent_kernel = rows
pc_backend = pmg_shell
```

```
load config → choose 3D SSR runner → load and reorder mesh
→ build owned elastic rows → prepare owned tangent pattern
→ create constitutive operator → build PETSc solver
→ launch indirect SSR continuation
```

I use this case to explain the current architecture. The committed P2 timings later answer a different question.

Config side

- `core/run_config.py` loads TOML, resolves relative paths, validates `elem_type`, analysis kind, materials, and public section shapes.
- The config object preserves the same high-level blocks MATLAB already used: problem, continuation, Newton, linear solver, and outputs.
- Material rows can be provided explicitly or inherited from the mesh-family definition.

```
cfg = load_run_case_config(...)
case = cfg.problem.case
runner, kwargs =
    _case_runner_kwargs(cfg)
result = runner(**kwargs)
write debug bundle
write history
write VTU
```

Policy inputs

- `solver_type`
- `mpi_distribute_by_nodes`
- `constitutive_mode`
- `node_ordering`

Default outcome

- owned tangent path enabled
- lightweight MPI elastic path enabled
- node numbering aligned to ownership through `blockmetis`

Why this matters

MATLAB kept one global sparse operator in one address space.

The PETSc rewrite makes row ownership explicit early, because that ownership controls:

- local element overlap
- constitutive work placement
- matrix assembly
- preconditioner construction

Owned Elastic Rows And A Fixed Tangent Pattern

Prepared once

- rank-owned node interval
- overlap elements for those rows
- local CSR matrix pattern
- elastic values on that pattern
- constrained diagonal positions

Refreshed every Newton step

- constitutive state on overlap integration points
- internal force rows
- tangent values on the fixed CSR pattern
- regularised matrix blend K_r

This is the deepest structural change relative to MATLAB: the rewrite no longer treats every Newton step as a fresh global sparse rebuild. The **structure** is prepared once; the **values** are refreshed in place.

Overlap constitutive mode

- Each rank evaluates the constitutive model on the overlap integration points needed by its owned rows.
- The default path avoids a unique-owner recovery stage.
- The tangent kernel receives local overlap data directly.

Rows kernel

- The kernel walks the fixed owned CSR pattern row by row.
- Metadata such as `row_slot_ptr`, `slot_elem`, `slot_lrow`, and `slot_pos` replace the old “assemble element matrix, then scatter” view.
- The bilinear form is unchanged; the dataflow is not.

What `pmg_shell` Means On This Mainline

- The fine-grid operator is the reduced free-space regularized operator coming from the owned-row tangent path.
- The outer Krylov family is still the PETSc-wrapped MATLAB-style deflated FGMRES interface.
- The actual preconditioner is a custom shell V-cycle, not the built-in PETSc PCMG path.
- The hierarchy metadata enters through `pmg_hierarchy` and stays aligned with the same reordered free-space numbering as the solve operator.

MATLAB intuition

one current sparse system plus one solver handle

PETSc mainline

current reduced operator plus shell hierarchy plus chosen outer KSP

When I say “parallel PMG” in this repository today, this shell-preconditioned reduced solve is the path I mean.

How The PMG Hierarchy Is Built And Rebuilt

runner startup:

build reordered pmg_hierarchy

default P4 chain: $P1 \rightarrow P2 \rightarrow P4$

store free-space levels + prolongations

Newton step n :

pass current reduced operator

validate level sizes + ownership

configure shell V-cycle on live matrix

- The hierarchy geometry and transfer metadata are built once from the reordered mesh family, element order, and material rows.
- `_ensure_pmg_state()` then turns those level-to-level transfers into PETSc matrices.
- The rebuild policy is still `every_newton`: the shell preconditioner is reconfigured on the current operator each Newton step.
- So the multilevel layout is stable, while the fine-grid matrix and smoother state stay current.

How The Outer Krylov Solve Sits On Top Of PMG

U, λ predictor

- refresh stress and tangent rows
- build reduced K_r
- outer deflated FGMRES sees A_f
- `pmg_shell` applies V-cycle
- correction returns to Newton

- `solver_type` still selects the outer MATLAB-style deflated FGMRES wrapper.
- `pc_backend = pmg_shell` selects the shell V-cycle that lives inside each outer Krylov step.
- The `mg_*` options then choose smoother behavior and the coarse-grid backend.

That is why I no longer read `solver_type` in isolation. The real linear solve lives in the combination of the outer wrapper and the PMG-shell backend under it.

What Changed Structurally To Enable Parallel Capabilities

Case spec extracted

Problem definition moved out of the script body and into TOML plus mesh-family metadata.

Ownership explicit

Node ordering, owned rows, and overlap elements are selected before the nonlinear loop starts.

Assembly split

Matrix structure is prepared once, while values are refreshed in distributed kernels.

Solve stack decoupled

Outer Krylov logic, preconditioner backend, rebuild policy, and exports are independent surfaces.

This is why the rewrite scales conceptually better than “MATLAB script + MPI wrapper”. The parallelism is in the native architecture, not around it.

Main New Functionality

MATLAB-parity suite

5 canonical parity cases are listed in `benchmarks/README.md`, each with MATLAB and PETSc timing summaries.

Additional runnable cases

The registry also carries LL, 2D, 3D, seepage, and SIOPT cases that are not part of the parity suite.

Shared metadata

Each folder uses the same [benchmark] contract, the same config runner, and the same notebook generation flow.

Once the folder contract exists, the shared registry becomes the cross-case surface: it lets me select suites, compare parity cases, generate notebooks, and automate over many benchmarks without changing the execution model.

Interface	Examples in the rewrite	Why it matters
[continuation]	predictor, omega controller, warm-start mode, fine switch, step-length cap	Continuation behavior is no longer hidden inside one script variant
[newton]	stopping criterion, stopping tolerance, damping parameters	Stop conditions are explicit and comparable across studies
[linear_solver]	backend, rebuild policy, Hypre/GAMG/BDDC options, recycle flag	Solver studies can be expressed in config instead of local script edits
[execution]	node ordering, ownership mode, tangent kernel	Parallel structure is public and reproducible

```
run case from config
→ save run_debug.h5
→ save continuation_history.json
→ save final_solution.vtu
→ save resolved_config.toml
→ reuse them in generated notebooks and external tools
```

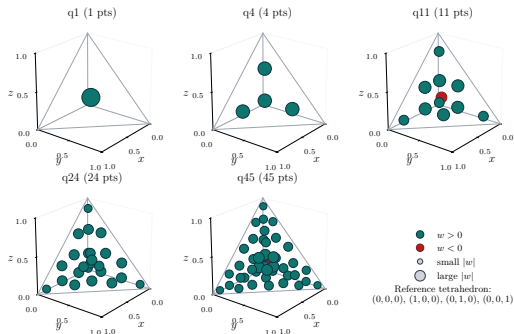
What this improves

- reproducibility
- postprocessing outside the solver process
- notebook regeneration without rerunning the case
- standard field names for PyVista and ParaView

MATLAB comparison

- MATLAB often kept these objects in workspace structs or figure state.
- The rewrite promotes them into stable file outputs.
- That is what enables the “unified visualisation” story later in the deck.

The Rewrite Also Carries A P4 Export And 3D Display Path



- The config and element helpers accept P1, P2, and P4 on the public interface.
- The VTK export path supports VTK_LAGRANGE_TETRAHEDRON for P4, including correct node ordering.
- The postprocess layer adds pointwise deviatoric strain export for 3D P4, so the higher-order benchmark path can be visualised without dropping back to a low-order fallback.
- I treat this as a concrete architecture capability exercised by the benchmark in this talk, not as a blanket maturity claim for every 3D runner.

Area	Mainline in this talk	Moved to appendix
3D mechanics path	P4 heterogeneous SSR default	legacy global assembly paths
Preconditioning	pmg_shell backend	BDDC, direct solves, built-in PETSc PMG variants
Continuation	indirect SSR, secant predictor, legacy omega control	direct SSR, fine-switch logic, streaming microsteps
Constitutive ownership	overlap mode	unique-owner recovery and legacy scatter variants
Speed study	committed P2 controlled report	local exploratory scaling archives

Unified Runners



```
benchmark_name/  
case.toml  
run.sh  
README.md  
simulation.ipynb  
visualisation.ipynb  
artifacts/simulation/...
```

- The folder, not the script file name, is the unit of execution and documentation.
- The same `case.toml` powers CLI runs, benchmark suites, generated notebooks, and export reconstruction.
- The folder layout is stable enough that automation can reason about every benchmark in a uniform way.

Section	Purpose
[problem]	Case identity, analysis kind, dimension, variant, element type, mesh path, Davis choice, seepage flag
[execution]	Node ordering, MPI ownership mode, constitutive ownership mode, tangent kernel
[continuation]	Predictor, omega controller, target horizon, delta-lambda rules, fine-switch and warm-start controls
[newton]	Iteration limits, damping limits, stop criterion, stop tolerance
[linear_solver]	Solver family, backend, matrix policy, rebuild policy, Hypre/GAMG/BDDC settings
[seepage]	Hydraulic solve tolerances and material conductivity data where relevant
[export]	Which standard outputs to write and how to name them
[[materials]]	Material rows in the same logical order as the MATLAB material table

```
[problem]
case = "3d_hetero_ssr"
analysis = "ssr"
elem_type = "P4"
mesh_path = "../meshes/...L1.msh"

[execution]
node_ordering = "block_metis"
constitutive_mode = "overlap"

[linear_solver]
pc_backend = "pmg_shell"
```

- This is enough to select the 3D SSR runner and the distributed architecture path I have been using as the anchor case.
- Notebook metadata lives in the same document, so the case also defines its 3D slice planes and display defaults.
- The config file is now the shortest route to understanding a benchmark.

load config

- inspect `problem.case`
- map to one capture runner
- convert config fields to runner kwargs
- run the case
- rebuild case mesh for export
- write debug bundle, JSON, VTU

- The dispatch layer is intentionally explicit rather than magical.
- Case-specific runners still exist, but the entrypoint surface is unified.
- This is the rewrite's replacement for choosing between many top-level MATLAB scripts.

How To Add A New Benchmark: Steps 1 To 3

1. Create a new folder under `benchmarks/` with `case.toml`, `run.sh`, and `README.md`.
2. Fill the `[benchmark]` block so the case has a title, MATLAB reference script, comparison kind, MPI rank default, and suite membership.
3. Fill the runtime blocks: `[problem]`, `[execution]`, `[continuation]`, `[newton]`, `[linear_solver]`, and optionally `[seepage]`, `[export]`, `[[materials]]`.

If the case belongs to an existing mesh family, reusing the mesh-family definition can remove the need to restate the material table in the benchmark itself.

How To Add A New Benchmark: Steps 4 To 5

1. Add an optional `[notebook]` block so the generated simulation and visualisation notebooks know the family, display backend, P4 surface subdivision, decimation, and slice planes.
2. Run the case once so `artifacts/simulation/` exists and the standard outputs can be reused by notebooks and postprocessing.

Checklist before you call it “done”

- Does `run.sh` call the same config that the notebooks use?
- Does the README describe the case without hiding the runtime contract?
- Are the exported fields sufficient for the visualisation family you selected?
- If it belongs in the parity suite, is `[benchmark].suite = true` set intentionally?

Unified Visualisation

solve case

- write final_solution.vtu + debug bundle + history JSON
- rebuild case mesh from config
- build standard field exports
- generated notebooks / PyVista / ParaView consume the same files

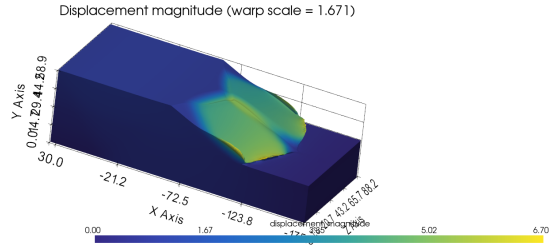
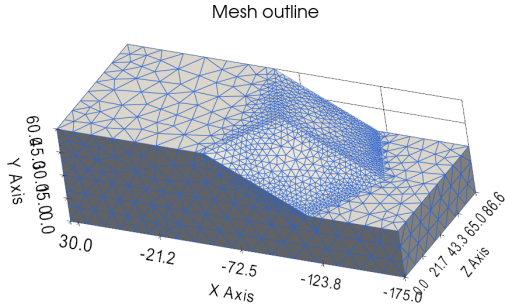
Key rewrite helpers

- postprocess/case_mesh.py
- postprocess/field_exports.py
- benchmarks/notebook_support.py

What that buys

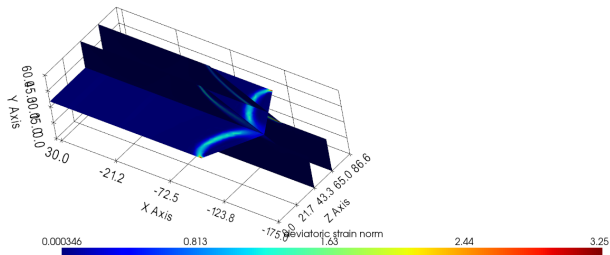
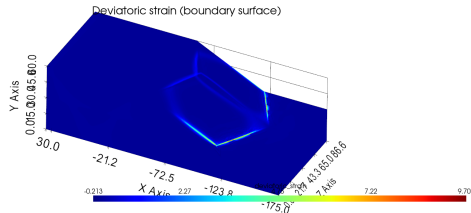
- one mesh reconstruction surface
- one field naming surface
- one notebook generation surface
- one export surface for external viewers

PETSc 3D Views: Geometry And Warped Displacement



- Both panels are taken from the default indirect 3D SSR visualisation notebook path.
- On the left I show the mesh-outline view that makes the local refinement pattern immediately visible.
- On the right I show the warped displacement view from the same benchmark and the same exported solution bundle.

PETSc Localisation Surface And Top-View Slices



- Both panels come from the same default indirect visualisation path and the same exported deviatoric_strain field.
- The 3D panel keeps the notebook-style full boundary-surface view that the authors are already used to reading.
- The slice panel stays a real top-view cut product at the benchmark-configured planes, with the same capped warm-to-cool scale used in the 3D deviatoric view shown on the left.

MATLAB And PETSc Reach Similar Slice Products Through Different Workflows

MATLAB

MATLAB: X = -71.01



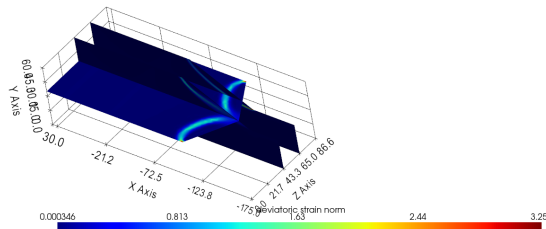
MATLAB: Y = 30.08



MATLAB: Z = 44.71



PETSc rewrite



What changes for me is not whether I can inspect the same slice family. What changes is where the slice product comes from. MATLAB builds it in the case script; the rewrite rebuilds it from shared exports and shared notebook support.

Output	Why it matters
<code>final_solution.vtu</code>	Standard mesh plus field export for PyVista, meshio, and ParaView
<code>run_debug.h5</code>	Dense debug bundle for deeper analysis and reproducibility
<code>continuation_history.json</code>	Lightweight branch history for dashboards and comparisons
<code>resolved_config.toml</code>	Exact runtime config that produced the outputs

- For 3D cases, the postprocess layer can rebuild the case mesh, attach displacement, pore pressure, saturation, and deviatoric strain, and then hand those fields to PyVista viewers or save them for ParaView.
- That is a much more uniform story than “run the script and hope the workspace still contains the right arrays for plotting”.

Unified Meshes

MATLAB Mesh Handling And Element Degree Are Entangled

Question	MATLAB answer	Consequence
Which file do I load?	Usually a degree-specific loader	Mesh storage and FE order drift together
How do I get P4 support?	Build new midpoint utilities and matching assumptions	Higher order grows sideways by utility proliferation
Where are materials and BC labels interpreted?	Inside the case loader path	Harder to reuse across benchmark families
Where does the free mask come from?	Loader-specific face conventions	Less separation between geometry and solve policy

```
DEFINITION = {  
  "name": "3d_hetero_ssr",  
  "storage": "gmsh_tet4_physical_groups",  
  "default_mesh": "SSR_hetero_ada_L1.msh",  
  "dirichlet_labels": {...},  
  "materials": [ ... ]  
}
```

- Each mesh family can declare canonical storage, default files, boundary labels, and material rows.
- The benchmark can then reference the family by mesh path and optionally inherit its material table.
- This makes the mesh family a reusable asset, not just a raw file on disk.

Canonical Gmsh Tet4 Storage With Loader-Side Elevation

canonical family mesh

- Gmsh tet4 + physical groups
- loader reads boundary/material tags
- solver requests P1, P2, or P4
- loader elevates connectivity as needed
- exports preserve the chosen FE order

- The family file does not need to be duplicated per degree.
- The rewrite can keep one canonical geometry/material definition while the solver selects its FE degree.
- This is the cleanest contrast to the MATLAB P2-only HDF5 path.

Mesh Family And `elem_type` Are Now Separate Concepts

Concept	Where it lives	Meaning
Mesh family	<code>meshes/*/definition.py</code> + mesh files	Geometry, physical tags, default material metadata
<code>problem.mesh_path</code>	<code>case.toml</code>	Which family member or external mesh file to load
<code>problem.elem_type</code>	<code>case.toml</code>	Which FE degree the runtime should use on that mesh
Export cell type	<code>core/elements.py</code>	How the chosen FE degree is written back to VTU

This is the core 3D mesh design change: **geometry/material family selection** and **finite-element degree selection** are no longer the same decision.

Boundary labels

Mesh-family metadata declares which physical labels correspond to constrained x , y , and z directions.

Material identifiers

Element-wise material ids stay attached to the mesh and can be expanded into integration-point rows later.

Node reorder

The same mesh can then be reordered for ownership without losing its physical meaning.

The rewrite does not hide physics labels behind generic partitioning. It keeps labels explicit, then reorders the discrete system around them.

Speed Comparison

Before I Quote Runtimes, Here Is The Locked Study Protocol

- The committed PETSc-vs-MATLAB performance report uses indirect SSR only and P2 elements only.
- PETSc runs use `mpirun -n 8` with `OMP_NUM_THREADS = 1`.
- MATLAB runs use `OMP_NUM_THREADS = 8` with the BoomerAMG MATLAB path on 8 threads.
- The three primary cases are homogeneous 3D SSR, heterogeneous 3D SSR, and heterogeneous 3D seepage SSR.
- The study is sequential and normalised into CSV before figures and tables are built.

The study answers “how does the committed P2 PETSc implementation compare under this controlled protocol?” It does **not** directly answer “how fast is the current default P4 architecture path?”.

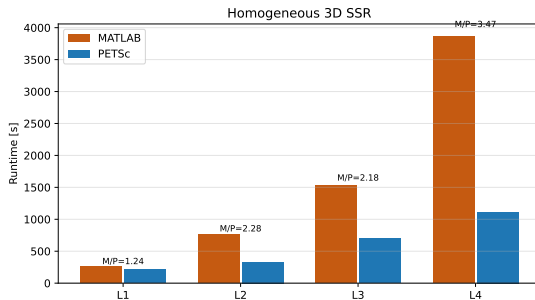
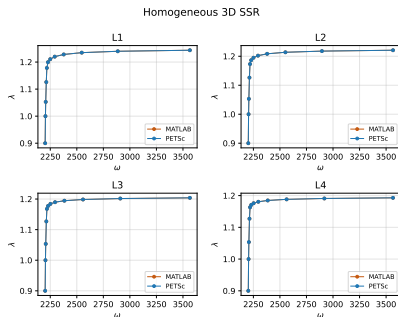
Source: docs/petsc_matlab_performance_comparison: README and main LaTeX report

Locked P2 Study: Headline View Across The Three Cases

Case	Levels	M/P gap	Headline
Homogeneous 3D SSR	L1 to L4	1.24× to 3.47×	PETSc pulls further ahead as the dry mesh ladder grows.
Heterogeneous 3D SSR	L1 to L4	1.35× to 3.08×	Same trend on the dry case closest to the architecture anchor.
Heterogeneous 3D seepage SSR	concave_L2 only	2.02×	Positive completed level, but the next level remains part of the reported limitation.

Source: committed P2 study generated tables in docs/petsc_matlab_performance_comparison/generated

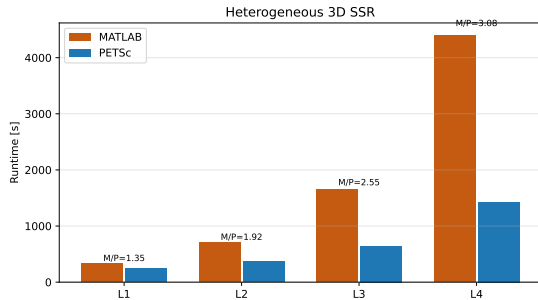
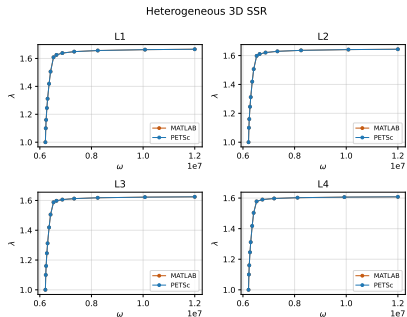
Homogeneous 3D SSR: PETSc Pulls Ahead As The Mesh Grows



- PETSc runtime goes from 210.5 s at L1 to 1115.2 s at L4.
- MATLAB runtime goes from 261.8 s at L1 to 3865.1 s at L4.
- The MATLAB-over-PETSc ratio grows from $1.24\times$ to $3.47\times$.

Source: presentation_petsc_rewrite/assets: homogeneous 3D continuation and timings from the committed study

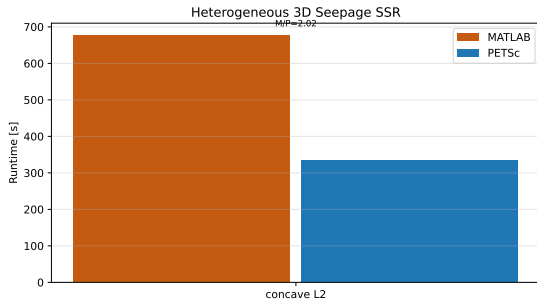
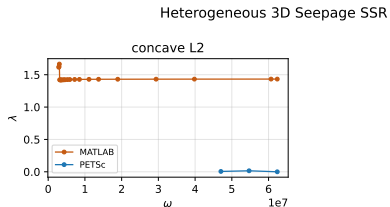
Heterogeneous 3D SSR: Same Trend, Stronger Advantage On Larger Levels



- PETSc runtime grows from 247.3 s at L1 to 1430.2 s at L4.
- MATLAB runtime grows from 334.2 s at L1 to 4399.1 s at L4.
- The MATLAB-over-PETSc ratio grows from $1.35\times$ to $3.08\times$.

Source: presentation_petsc_rewrite/assets: heterogeneous 3D continuation and timings from the committed study

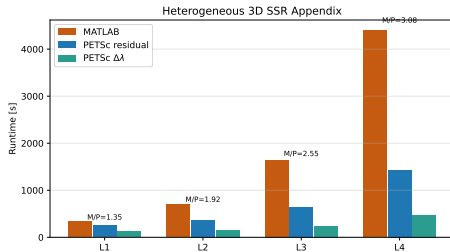
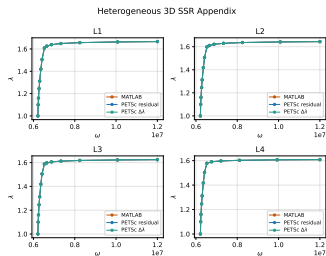
Seepage 3D SSR: Only One Level Completed Under The Locked Protocol



- On concave_L2, PETSc finished in 334.1 s and MATLAB in 676.3 s, a $2.02\times$ advantage.
- Only this level completed under the committed mixed PMG-shell seepage study protocol.
- The next seepage level failed on the PETSc side after hierarchy construction and outer-solver setup, so the report deliberately stops at the completed comparison.

Source: presentation_petsc_rewrite/assets: seepage 3D continuation and timings from the committed study

Additional Speedup Gain: The Delta-Lambda Stop Rule



- On the same heterogeneous dry study, the $\Delta\lambda$ stop rule is $2.0\times$ faster than the residual baseline at L1 and $3.0\times$ faster at L4.
- Against MATLAB, that pushes the gap from $2.7\times$ at L1 to $9.3\times$ at L4.

I treat this as a controlled “other speedup gain” slide. The study branch stays the same; only the stop policy changes.

Source: docs/petsc_matlab_performance_comparison: delta-lambda figures and generated table

Close

Key Takeaways For The Original MATLAB Authors

- The rewrite preserves the familiar nonlinear story, but it changes how responsibilities are packaged and where data lives.
- Config files and mesh-family metadata replace large parts of the old top-level script surface.
- Owned-row distributed assembly and explicit solver backends are the main structural changes that enabled parallel capabilities.
- Standard exports and notebook tooling replace much of the old case-local visualisation burden.
- The committed P2 speed study already shows clear PETSc advantages on the main 3D dry cases, while also documenting current seepage limitations honestly.

Recommended Reading Order For New Contributors

If your question is...	Start here
How does the default P4 architecture benchmark run end to end?	benchmarks/slope_stability_3D_hetero_SSR_default/case.toml, then cli/run_case_from_config.py, then cli/run_3D_hetero_SSR_capture.py
How does the old MATLAB logic map onto the rewrite?	docs/petsc_matlab_map/*
How are mesh family and materials declared now?	meshes/*/definition.py plus problem_assets.py
How are fields exported and visualised?	postprocess/*, benchmarks/notebook_support.py
What performance claims are currently documented?	docs/petsc_matlab_performance_comparison/*

What stayed the same

SSR, Newton, constitutive reduction, and the benchmark families remain recognisable to the MATLAB authors.

What changed

Configuration, ownership-aware assembly, solver backend separation, standard exports, and shared notebooks became first-class interfaces.

What to discuss next

Which paths should be treated as production defaults, which appendix branches deserve more work, and where parity versus performance should be prioritised next.

Optional Paths, Caveats, And Additional Reading

Architecture Mainline Versus Performance Study Protocol

Question	Architecture mainline	Committed performance study
Purpose	Explain the current rewrite structure	Compare PETSc and MATLAB under one locked protocol
Case anchor	default P4 benchmark	committed P2 report
Element degree	P4 default path	P2 only
Solver backend focus	pmg_shell, owned-row path	PETSc MPI-8 versus MATLAB OMP-8
What it proves	How the rewrite is structured today	How one committed PETSc branch compares numerically and in runtime

Optional And Inactive Paths Present In The Repository

- Direct SSR and LL continuation remain present and runnable for selected cases.
- BDDC, Hypr, GAMG, and multiple PMG variants all exist as selectable branches.
- Different constitutive ownership modes remain available beyond the overlap mainline.
- The map document keeps these paths in its annex so they do not distort the default-reading story.

Path	Still present?	Why it is not the default story
Global B' D B rebuild	yes	It matches the old MATLAB intuition better, but it is not the normal owned-row PMG route
Legacy tangent kernel	yes	Kept for comparison and fallback, but the rows kernel is the production default for the mainline path
Legacy scatter-style assembly	yes	Appendix material now that overlap-owned assembly is the default architectural choice

Branch	Repository status			Typical reason to look at it
hypre backend	active			Classical AMG comparison against PMG-shell
gamg backend	active			PETSc-native algebraic multigrid studies
bddc backend	active			Domain decomposition experiments and MATIS-style subdomain work
DIRECT	active mainline	but	non-	Small-case validation, not distributed production solves

- The seepage section reports only `concave_L2`.
- The next seepage level reproduced a PETSc-side crash after hierarchy construction and outer-solver setup in the mixed shell hierarchy.
- Same-mesh fallback diagnostics avoided the crash but did not yield a usable converged benchmark within the locked study protocol.
- The report therefore keeps the completed comparison and states the limitation instead of smoothing it away.

Level	PETSc residual [s]	PETSc $\Delta\lambda$ [s]	MATLAB [s]	Step counts
L1	247.304	122.460	334.248	14 / 14 / 14
L2	365.262	152.865	700.772	14 / 14 / 14
L3	645.949	239.822	1645.273	14 / 14 / 14
L4	1430.224	473.999	4399.089	13 / 13 / 13

Read this appendix as a protocol sensitivity note, not as a replacement for the main report.

Extra Source Map And Reading Order

1. docs/petsc_matlab_map/petsc_to_matlab_solution_map.tex
2. docs/config_scheme_3d.md
3. README.md
4. src/slope_stability/cli/run_case_from_config.py
5. src/slope_stability/cli/run_3D_hetero_SSR_capture.py
6. src/slope_stability/fem/distributed_tangent.py
7. src/slope_stability/linear/solver.py
8. benchmarks/notebook_support.py

If a MATLAB author wants the fastest path to orientation, the map document plus the default P4 architecture benchmark config are the highest-yield entry points.